

Software Design Principles

Presented by: Dr. Abdolraheem Khader
May 2025

Introduction to Software Design Principles

- **Definition:** Guidelines for creating robust, maintainable, and efficient software.
- **Objective:** Achieve code quality through structured design.
- **Importance:** Reduces complexity and enhances collaboration.

Key Principles in Software Design

- Modularity: Breaking down the system into manageable components.
- Abstraction: Hiding complex implementation details.
- Encapsulation: Restricting access to certain components.
- Reusability: Maximizing code usage across projects.

Modularity: Concept and Benefits

- **Definition:** Decomposing a system into independent, interchangeable modules.
- **Benefits:**
 - Improved maintainability
 - Enhanced scalability
 - Easier testing and debugging
- **Example:** Dividing an e-commerce application into user management, inventory, and payment modules.

Modular Design Principles

- **Cohesion:** Each module should perform a single, well-defined function.
- **Coupling:** Minimize dependencies between modules.
- **Separation of Concerns:** Each module addresses a specific aspect.
- **Example:** MVC (Model-View-Controller) architecture.

Modularity in Object-Oriented Design

- Techniques: Classes, Interfaces, Packages.
- Example: Designing a library system with separate classes for books, users, and transactions.
- Advantage: Modules can evolve independently.

Modularity in Functional Design

- **Techniques:** Pure functions and higher-order functions.
- **Example:** A functional pipeline for data processing.
- **Advantage:** Functions are self-contained and composable.

Abstraction: Concept and Importance

- **Definition:** Hiding the implementation details while exposing functionality.
- **Benefits:**
 - Reduces complexity.
 - Enhances code clarity.
 - Promotes flexibility.
- **Example:** Abstracting a file system API to work with both local and cloud storage.

Abstraction in OO Paradigm

- **Mechanisms:** Abstract classes and interfaces.
- **Example:** Shape class with derived classes like Circle and Rectangle.
- **Advantage:** Promotes polymorphism and code reuse.

Abstraction in Functional Paradigm

- **Mechanisms:** Higher-order functions and functional interfaces.
- **Example:** Mapping a list of numbers to their squares.
- **Advantage:** Simplifies complex operations using function composition.

Encapsulation: Concept and Benefits

- **Definition:** Bundling data with methods that operate on that data.
- **Benefits:**
 - Data protection
 - Controlled data access
 - Promotes data integrity
- **Example:** Private attributes in a Python class.

Encapsulation in OO Design

- **Mechanisms:** Private, protected, and public access modifiers.
- **Example:** BankAccount class with private balance attribute.
- **Advantage:** Prevents unauthorized access to data.

Encapsulation in Functional Design

- **Technique:** Closures to protect variables.
- **Example:** A function returning another function with a local variable.
- **Advantage:** Scope control without exposing data.

Reusability: Concept and Significance

- **Definition:** Using existing software components in new applications.
- **Benefits:**
 - Reduces development time
 - Increases consistency
 - Minimizes redundancy.
- **Example:** Using a library for data validation in multiple applications.

Reusability in OO Paradigm

- **Techniques:** Inheritance and polymorphism.
- **Example:** Base class Vehicle with derived classes Car and Bike.
- **Advantage:** Reduces code duplication.

Reusability in Functional Paradigm

- **Techniques:** Higher-order functions and reusable pipelines.
- **Example:** A function that filters even numbers from a list.
- **Advantage:** Modular and composable code.

Best Practices for Applying Design Principles

- **Follow SOLID Principles:** Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.
- **DRY Principle:** Don't Repeat Yourself.
- **KISS Principle:** Keep It Simple, Stupid.

Object-Oriented Design Patterns

- **Creational:** Singleton, Factory.
- **Structural:** Adapter, Decorator.
- **Behavioral:** Observer, Strategy.
- **Example:** Using the Factory pattern for object creation

Functional Design Patterns

- **Immutable Data:** Avoids side effects.
- **Function Composition:** Combining simple functions to build complex logic.
- **Monads:** Handling computations with context (e.g., error handling).
- **Example:** Using Monads to manage optional values.

Real-World Example: E-commerce System

- **Modularity:** Product, User, and Order modules.
- **Abstraction:** Interface for payment gateway.
- **Encapsulation:** Private methods for transaction processing.
- **Reusability:** Product filtering logic as a reusable function.

OO vs. Functional Design

- **OO:** Emphasizes objects and their interactions.
- **Functional:** Emphasizes pure functions and immutability.

Common Challenges in Software Design

- Balancing Modularity and Performance.
- Excessive Abstraction.
- Over-Encapsulation.
- Lack of Reusability in tightly coupled designs

Design Principle Violations (Anti-Patterns)

- **God Object:** Overloaded class doing too much.
- **Spaghetti Code:** Poor modularity.
- **Lack of Encapsulation:** Data exposed without protection.
- **Example:** An unstructured monolithic application.

Case Study: Social Media Application

- **Problem:** Integrating new features without breaking existing code.
- **Solution:** Modular design with abstraction layers.
- **Outcome:** Improved maintainability and scalability.

Tools for Software Design

- **UML Modeling:** Enterprise Architect, Lucidchart.
- **Code Analysis:** SonarQube.
- **Code Reusability Check:** CodeClimate.
- **Version Control:** Git, SVN.

Future Trends in Software Design

- **Microservices:** Modular service-oriented architecture.
- **Serverless Functions:** Modular and scalable function deployment.
- **Design Patterns for AI Systems:** Handling data pipelines.

Workshop Activity

- **Task:** Design a modular calculator using both OO and functional approaches.
- **Discussion:** Compare modularity and reusability between paradigms.

Thank You!